Study Material — Part 5: Prompt Engineering, LLM & Embedding Model Selection, Application Architecture

Deliverable 4 (Part 5 of several) — Databricks GenAI Engineer Associate

This covers the remaining Section 1, 3, and 4 objectives: how to engineer prompts that actually work, how to select the right model for the right task, and how to architect LLM applications correctly on Databricks using frameworks like LangChain.

Topic 26: Prompt Engineering Fundamentals

What is it?

Prompt engineering is the practice of designing the text input you send to an LLM — the question, instructions, examples, and context — to reliably produce the output you want, without modifying the model's weights.

Why does it matter for the exam?

The exam tests prompt engineering from two angles:

1. **When is prompt engineering the right tool** (vs. fine-tuning, vs. RAG, vs. a different architecture)?
2. **What specific techniques work for specific problems?**

Core prompt engineering techniques

Zero-shot prompting

Give the LLM the task and ask it to perform it with no examples.

```
"Summarize the following product review in one sentence: [review text]"
```

Works well when the task is simple and within the model's training distribution. Fast, cheap, but less reliable for nuanced or non-standard outputs.

Few-shot prompting (in-context learning)

Provide 2–5 examples of the desired input→output pattern before the actual task.

```
"Input: 'Great product, would buy again!' → Sentiment: Positive
Input: 'Broke after one day, total waste of money.' → Sentiment: Negative
Input: '[new review text]' → Sentiment:"
```

Dramatically improves performance on classification, extraction, and formatting tasks without any fine-tuning. Key insight: the model learns the pattern from your examples at inference time — no training step needed.

Chain-of-Thought (CoT) prompting

Ask the model to "think step by step" before giving a final answer.

```
"Solve this problem step by step, then give your final answer: [math word
problem]"
```

Improves accuracy on multi-step reasoning tasks (math, logic, planning). Important: CoT is **reasoning only** — it doesn't enable tool calls. Don't confuse it with ReAct (which does).

System prompt (role + constraints)

Use the system message to set the LLM's persona, scope, and behavioral constraints before the user's question.

```python
system_prompt = """You are a professional technical support agent for Acme Corpora
- Answer ONLY questions about Acme products.
- Always respond in formal English.
- Never speculate. If you don't know, say 'I don't have that information.'
- Always end with: 'Is there anything else I can help you with?'"""
```

Formatting instructions

Tell the LLM exactly what format you want the output in.

```
"Respond with a valid JSON object with keys: 'subject', 'priority' (1-5),
'summary'. No extra text."
```

Important for downstream parsing — if your code needs to parse the LLM's output, give explicit format instructions rather than hoping the model "knows" what you need.

Tone and style instructions (directly tested in Q33)

```
"Write the following headline in a witty, conversational tone suitable for a
tech startup blog.
Avoid corporate jargon. Keep it under 10 words. Article content: [content]"
```

This is the **most direct, lowest-cost fix** for a tone/style mismatch — just describe the desired style explicitly in the prompt. Fine-tuning is disproportionate and slow for this class of problem.

When prompt engineering is the right tool (vs. alternatives)

Problem	Best solution	Why
LLM produces wrong format	Prompt: add format instructions	Fast, cheap, no training
LLM has wrong tone/style	Prompt: add style instructions	First-line fix; fine-tuning only if prompting fails
LLM doesn't know your proprietary data	RAG	Can't prompt in knowledge the model never saw
LLM needs to behave very differently from its defaults	Fine-tuning	Deep behavioral change requires weight adjustment
LLM needs to perform a task it struggles with even with examples	Fine-tuning or a task-specialized model	Prompt can't compensate for a fundamental capability gap

Preprocessing prompts with custom code (directly tested in Q1)

When you need to run custom Python logic before sending a prompt to an LLM:

- **Wrap the preprocessing logic in an MLflow `pyfunc` model** — this makes it versioned, deployable, testable, and part of the served pipeline
- The `pyfunc` wraps both the preprocessing step AND the LLM call into a single deployable unit
- You do NOT modify the LLM's architecture to add preprocessing — you add it as a separate step in the serving pipeline

Important exam points

- "Tone/style doesn't match" = **fix with prompt instructions first** (never fine-tune as a first resort for this)
- "Producing the wrong format" = **add explicit format instructions** (not post-process the output as a workaround)
- "Custom preprocessing before the LLM call" = **MLflow pyfunc wrapper** with preprocessing function

- Few-shot examples in the prompt ≠ fine-tuning — the examples are just part of the text input; no weight updates happen

Common mistakes/traps

- Reaching for fine-tuning when a well-crafted prompt would have worked (classic "too much gun for the problem")
- "All of the above" answers for prompt-engineering questions — often wrong when one of the listed options (like fine-tuning) is disproportionate for the stated problem

---

Topic 27: LLM Selection

What is it?

Choosing the right language model for a given task — from among open-source models (available on HuggingFace, Databricks Model Serving, etc.) and closed-source API models — based on the actual requirements of the task.

The correct selection process (directly tested in Q38)

```
Step 1: What modality is needed?
         → Text generation? Speech-to-text? Image understanding? Code?
         → Immediately eliminates most models (a text LLM can't do speech-to-
text)

Step 2: What domain or specialization is needed?
         → Medical? Legal? Code? General?
         → Check model card / training data description for domain coverage
         → Domain-trained > general model for specialized tasks

Step 3: What are the constraints?
         → Latency? Cost? Size? Privacy (on-premise)?
         → Check model size, context length, serving requirements

Step 4: Evaluate empirically
         → Don't pick based on brand, download count, or leaderboard rank alone
         → Test on a representative sample of YOUR actual data and queries
```

Key model categories the exam tests

General instruction-tuned LLMs

Models fine-tuned to follow natural-language instructions: Llama (Meta), Mistral, DBRX (Databricks), MPT.

- Named with suffixes like `-instruct`, `-chat`, `-hf`
- Good for general text generation tasks

Code-specialized LLMs (directly tested in Q10)

- **CodeLlama** — Meta's code-focused model fine-tuned from Llama, with versions in 7B, 13B, 34B parameters
- **CodeLlama-34b-Instruct-hf** = instruction-tuned, 34B parameter code model → best "out of the box" code generation in the options given in Q10
- Don't default to "biggest = best" — a smaller model *specialized for code* outperforms a larger general model at code generation

Speech-to-text models (directly tested in Q28)

- **Whisper** (OpenAI) — purpose-built for automatic speech recognition (ASR / transcription)
- `whisper-large-v3` = large, high-accuracy variant; `whisper-small` = faster, smaller
- Llama/Mistral/DBRX are text models — they cannot transcribe audio. This is a modality mismatch, not a size difference.

Embedding models

See Topic 3 in Part 1. Key: match to domain, don't just pick highest MTEB rank.

Selecting a model from HuggingFace (directly tested in Q38)

The correct selection criterion: **check the model card and training data description** to confirm:

1. Was the model trained on data relevant to my domain? (e.g., for medical documents: was it trained on medical literature?)
2. What are the license terms?
3. What task was it fine-tuned for?

Wrong criteria (all explicitly tested as distractors):

- "Pick the one with the most downloads" — download count measures popularity, not domain fit
- "Pick the most recently uploaded" — recency doesn't indicate quality or relevance
- "Pick any model in the [Family] family, they're all good for everything" — a model family can have general-purpose and specialized variants; "all good for all use cases" is never true

Important exam points

- Modality mismatch eliminates most options immediately in model-selection questions (a text LLM cannot do speech transcription, regardless of size)
- Model card / training data inspection = the correct research step for specialized domain tasks
- Popularity, recency, and family membership are all red-herring criteria for task-specific selection

---

Topic 28: Embedding Model Context Length Selection (Official Study Gap)

What is it?

Every embedding model has a maximum input length it can accept — its **context length**, measured in tokens. If your chunk exceeds this, the model either truncates the chunk silently (losing content) or throws an error.

Why it's its own tested objective

Choosing an embedding model isn't only about *which model* (domain fit, MTEB rank) — it's also about matching the model's context length to your chunking strategy AND your cost/latency/quality constraints. These are separate decisions.

The decision framework (directly from official sample Q4)

```
Constraint 1: Max chunk size = 512 tokens (set by your chunking strategy)
Constraint 2: Cost and latency matter more than quality (explicit in scenario)

Which embedding model do you pick?
→ The SMALLEST model whose context length still covers 512 tokens
→ Larger context length = larger model = more memory, more compute, more
latency, more cost
→ If 512 tokens is the max you'll ever embed, a 1024-token-context model is
already 2x more
   expensive than a 512-token-context model with no quality benefit for your
data

Counter-scenario: if quality matters more than cost, pick the strongest model
whose
context length covers your chunks — MTEB rank then becomes a relevant secondary
signal.
```

Embedding dimension and its tradeoffs

	Low embedding dimension (e.g., 384)	High embedding dimension (e.g., 4096)
Storage	Less (smaller vectors)	More
Similarity search compute	Faster	Slower
Semantic richness	Less nuance	More nuance (potentially)
Cost	Lower	Higher

For most tasks, dimensions in the 768-1536 range hit the quality/cost sweetspot. Only use very high dimensions if you have a strong empirical reason — don't over-provision.

Important exam points

- Context length selection = match to your **largest chunk size** — never let your chunks be larger than the embedding model's context window
- Cost/latency priority → **smallest model that covers your chunk size**
- Quality priority → stronger model with appropriate context length, informed by MTEB and domain testing

---

Topic 29: LangChain and Multi-Step LLM Workflows

What is it?

LangChain is a popular open-source Python framework for composing multi-step LLM-powered workflows — chains, RAG pipelines, agents, memory management, and tool use. Databricks recommends and supports LangChain (and LangGraph for stateful agents) as a primary framework for building GenAI applications.

Why it's exam-relevant

Section 3 objective: "Select LangChain or similar framework for a multi-step LLM-based workflow." The exam tests whether you know LangChain is the right framework for *composing* LLM workflows — not PySpark (data processing), TensorFlow (model training), or Pandas (data manipulation).

Core LangChain concepts for the exam

Chains

A sequence of components (LLM calls, retrievers, memory, tools) wired together. The simplest chain: prompt → LLM → output parser.

```python
from langchain.chains import LLMChain
from langchain.prompts import ChatPromptTemplate

chain = ChatPromptTemplate.from_template("Summarize: {text}") | llm | output_parse
result = chain.invoke({"text": document_text})
```

Memory components (directly tested in Q19)

- `ConversationBufferMemory` — stores the full conversation history in memory; passed to the LLM on each turn so it "remembers" previous messages
- This is the "contextual memory" component for a basic multi-turn chat application

Vector Store integration (directly tested in Q19)

- LangChain's `VectorStoreRetriever` wraps a vector database (e.g., Databricks Vector Search) as a retrieval component in a chain
- This is the "knowledge retrieval" component

Basic LLM-enabled chat application components (directly tested in Q19)

For a basic chat app with conversational capability + knowledge retrieval:

- **Vector Store** (knowledge retrieval) ✅
- **Conversation Buffer Memory** (contextual memory for multi-turn) ✅
- External tools ❌ (not needed for "basic" — adds agentic complexity)
- ReAct Components ❌ (not needed for "basic" — that's an agent, not a chain)
- Chat loaders ❌ (for importing prior conversation data, not a live chain component)

LangGraph (for agents)

LangGraph is LangChain's graph-based framework for building stateful, multi-step agents — supports cycles (looping back for more tool calls), conditional branching, and state persistence (checkpointing to Lakebase or similar).

Important exam points

- "Multi-step LLM-based workflow" = LangChain (or similar — LlamaIndex, DSPy are alternatives; the exam mainly tests LangChain)
- "PySpark" is for Spark data processing on large datasets — not for LLM orchestration
- "TensorFlow/PyTorch" is for model training — not for LLM application orchestration

---

Topic 30: Designing Input/Output Pairs for LLM Tasks (Section 1 Objective)

What is it?

Before you build an LLM pipeline, you need to define precisely what the input to the LLM will be and what the output should be — translated from a business requirement into a concrete ML task definition.

Why it matters

An incorrectly defined input/output pair results in a model that produces the wrong type of output (even if it's a technically good LLM performing that wrong task correctly). The exam tests whether you can match a business requirement to the right LLM task formulation.

Common input/output patterns

Business need	Input	Output
Sentiment-adaptive support bot	Customer message (→ classify sentiment first, then)	Response conditioned on sentiment
Personalized retail experience	Customer chat history	Per-user grouped summary → personalized response
Document Q&A	Retrieved chunks + question	Grounded answer
Information extraction from PDFs	Raw document text	Structured JSON (name, date, product, etc.)
Ticket root-cause analysis	Call transcript text	Extracted fields (subject, resolution, root cause)
Translation	Source language text	Target language text
Summarization	Long document	Short summary

Task decomposition (multi-step)

When one LLM call can't handle the full requirement:

1. **Identify what the LLM needs to "know" at each step** (e.g., to respond based on sentiment, you must know the sentiment first)
2. **Chain steps so earlier outputs become later inputs** (step 1: classify sentiment → step 2: use sentiment in system prompt for generation)
3. Avoid putting steps 1 and 2 in a single LLM call when the step-1 output needs to deterministically alter the step-2 prompt — chains give you that control; a single call doesn't

Directly tested in Q24 and Q49

- Q24: retail chatbot — need to group customer reviews by user, aggregate ratings per user, then respond to individual questions → correct input/output pair: customer reviews → per-user summary → response
- Q49: sentiment-adaptive support → correct chain: first LLM call classifies sentiment → result feeds system prompt → second LLM call generates appropriately-toned response

---

Topic 31: Compound AI System Design — Routing & Multi-Source Workflows

What is it?

A compound AI system combines multiple models, retrievers, tools, or data sources to handle a task no single model/pipeline could handle well alone. The key architectural challenge: **routing** — deciding which component should handle which type of request.

Why it matters

Several of your harder scenario questions (Q25, Q53) describe systems that must handle different types of queries with different tools or data sources. The exam tests whether you can identify the right architectural pattern.

Key routing patterns

Single-source approach (preferred when possible)

If all query types can be served by one unified retrieval mechanism, use one RAG pipeline with a common document format. Less complexity, easier to evaluate and maintain.

```
Problem: users ask both "what's the historical failure rate for part X?"
         AND "how do I troubleshoot part X not powering on?"

Both types → RAG over a document corpus that includes:
- A table of failure rates imported and chunked as text
- The troubleshooting manual chunked and indexed

One RAG pipeline handles both.
```

Multi-step routing (for fundamentally different data types)

When different query types need different tools/backends, route first, then direct to the appropriate handler:

```
Intent classifier → structured query → text-to-SQL model → database result
                  → unstructured query → RAG pipeline → document answer
```

Multi-source data selection (directly tested in Q25)

When you have many candidate data sources, select the ones whose *actual described content* matches the information need — not the ones that *sound* like they might be relevant. Read the field descriptions carefully.

Key Q53 teaching point

The scenario has two question types (historical failure rates from a table, equipment troubleshooting from PDFs). The correct answer avoids building two separate chatbots by converting the structured table data into a format that can be served by the same document-retrieval pipeline. The exam favors the *simplest architecture that satisfies all requirements* over the *most sophisticated* one.

Quick self-check — all of Part 5

1. Name three prompt engineering techniques and describe one situation where each is the right choice.
2. Why is fine-tuning the wrong first fix for a tone/style mismatch?
3. How do you wrap custom preprocessing logic for an LLM call in Databricks?
4. What's the first thing to eliminate when selecting a model (before checking size, rank, or downloads)?
5. What embedding model context length should you choose if your largest chunk is 512 tokens and cost/latency are the priority?

6. Name two LangChain components needed for a basic multi-turn RAG chat app.
7. What's the difference between a chain and a multi-step routing system?
8. Why does "most downloads on HuggingFace" not reliably indicate the best model for a specialized task?

Next: Study Material Part 6 (Final Part) — The Exam Cheat Sheet: concise tables, comparisons, memory tricks, "remember this" points, and the most commonly confused concept pairs — everything you need for last-day revision.